



M.C.E. Society's

Abeda Inamdar Senior College

Of Arts, Science & Commerce, Pune

Affiliated to Savitribai Phule Pune University

A Field Project Report On

SMART HEALTHCARE SERVICES PLATFORM

SUBMITTED BY

Shahid Sayyad

Pathan Mazz

Under the Guidance of

Veena Gandhi

TYBSc(CA)

This Field Project Report is hereby presented to the esteemed Department of Computer application, signifying the satisfactory completion of the requirements for the Sem V of Bachelor of Computer Application during academic year 2026-2027.



M. No. : PU / PN / ASC /
086 / 1992

M.C.E. Society's

ABEDA INAMDAR SENIOR COLLEGE of Arts, Science & Commerce (Autonomous), Pune

Affiliated to Savitribai Phule Pune University | (Accredited with 'A' grade by NAAC)

Prof. Dr. Shaila Boothwala (M.Com., M. Phil., Ph.D.)
Principal

Mrs. Abeda P. Inamdar
President, M.C.E. Society, Pune



Ref. AISC /

Date :

Department of B.Sc (Computer Application)

Certificate

This is to certify that **Mr./Ms.** _____, a
student of **BSC(CA)** of this college, has **successfully completed** his/her **Field
Project (FP)** titled _____ as
prescribed in the curriculum of **Sem _____**, during the academic year **20_____ – 20_____**.

During this program, the student's performance, conduct, and dedication were found to
be **satisfactory**.

Project Guide

Head, Department of B.C.A (Science)

Internal Examiner

External Examiner

[this print have to take on company's letterhead with the authorized signature]

This is to certify that,

Mr./Ms. Shahid Sayyad, a student of **Division B, Department of Computer Application, Abeda Inamdar Senior College, Pune**, has successfully completed a **Field Project** at

The student actively participated in the **study, observation, and analysis of the system/work processes** and gained practical knowledge in the field of **[Domain/Area, e.g., Business Processes, IT Systems, Field Survey, etc.]**.

The project work carried out by the student has been found satisfactory and in line with the objectives of the field project.

[Authorized Signature]

Index

SR NO.	TOPIC	PAGE NO.
1	Introduction	
2	Problem Definition	
3	Current System	
4	Proposed System - Purpose - Scope & Objectives - Functional Requirement - Nonfunctional Requirement	
5	Need of the Proposed System	
6	Scope of the Proposed System	
7	Fact Finding Techniques	
8	Feasibility Study	
9	Hardware and Software Requirements	
10	ER Diagram	
11	Data Dictionary	
12	DFD/UML Diagrams	
13	Input/Output Screens	
14	Limitations	
15	Conclusion	
16	Bibliography	

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported and guided us in completing our Final Project titled “**Smart Healthcare Services Platform**”.

First and foremost, we would like to thank our Project Guide, **Vina Mam**, for providing valuable guidance, suggestions, and continuous support throughout the development of this project.

We are grateful to the Head of the Department, **Vina Mam**, and Abeda Inamdar Senior College, Pune, for providing us with the necessary facilities and encouragement to complete this project successfully.

We would also like to thank all the faculty members of the Department of BCA / Computer Application for their valuable support and suggestions during the project.

Finally, we express our heartfelt thanks to our parents, friends, and classmates for their encouragement and support throughout the completion of this project.

Thank You.

Project Team

Shahid Sayyad and Mazz Pathan

INTRODUCTION

The **Smart Healthcare Services Platform** is a multi-tenant web application designed to streamline administrative operations, facilitate doctor-patient communication, and enhance overall healthcare accessibility. Integrated with an intelligent AI health assistant, the system offers personalized health assistance and automated support. The system supports three primary user roles: Patients, Doctors, and Administrators. Key capabilities include seamless appointment booking, doctor schedule management, secure payment processing, and interactive AI chatbot assistance. The application is built using modern web technology stacks comprising React for frontend development, FastAPI for the backend framework, and MongoDB for database management.

PROBLEM DEFINITION

Conventional hospital administration relies heavily on fragmented, manual, or paper-based record systems. These processes introduce substantial operational inefficiencies, including prolonged appointment waiting times, scheduling conflicts, and inaccurate record keeping. Furthermore, the lack of centralized digital touchpoints hinders effective patient engagement and delays medical consultations.

EXISTING SYSTEM

The current clinical workflow primarily depends on manual registers, direct phone calls, or physical front-desk visits to arrange consultations and manage records. This traditional approach suffers from significant limitations in efficiency, accessibility, and centralization. Data is often scattered across distinct paper files or unintegrated software tools, increasing administrative overhead and susceptibility to human error.

PROPOSED SYSTEM

The proposed Smart Healthcare Services Platform provides a unified digital ecosystem designed to resolve operational bottlenecks in clinical administration while modernizing doctor-patient interaction.

Purpose

The primary purpose of the proposed system is to centralize hospital administration and digitize healthcare workflows. It enables secure user interaction, streamlined scheduling, automated payment processing, and instant AI-assisted health guidance.

Scope & Objectives

The scope encompasses digital user registration and verification, end-to-end appointment scheduling, interactive AI medical assistant integration, and doctor review functionality. However, role-based access control, advanced doctor search filtering, and appointment rejection workflows are currently only partially implemented. Real-time slot locking, telemedicine integration, and automated test coverage are out of scope for the current version.

Functional Requirements

FR-01: The system shall allow users to register using personal information.

FR-02: The system shall verify user registration using email OTP.

FR-03: The system shall allow patients to browse doctors and schedule appointments.

FR-04: The system shall support secure Razorpay payment processing for appointment fees.

FR-05: The system shall provide an AI health assistant chatbot for user inquiries.

Non-Functional Requirements

Performance: The application shall respond to API requests within optimal latency limits to ensure smooth user interactions.

Security: Sensitive user data and authentication processes shall be encrypted and protected against unauthorized access.

Usability: The user interface shall be intuitive, responsive, and easy to navigate for patients and administrative personnel.

Maintainability: The modular React and FastAPI architecture shall allow straightforward updates and codebase extensions.

Reliability: The system shall ensure consistent data integrity across database interactions

and payment gateway communications.

NEED OF THE PROPOSED SYSTEM

The traditional healthcare administrative environment often suffers from significant operational bottlenecks, including reliance on manual paper-based record-keeping, disparate information systems, and inefficient communication channels between patients and healthcare providers. The "SMART HEALTHCARE SERVICES PLATFORM" is proposed to address these critical inefficiencies by providing a centralized digital ecosystem.

The system is needed for several reasons:

- **Centralized Data Management:** It replaces fragmented manual records with a unified digital database, ensuring data integrity and accessibility across hospital operations.
- **Improved Scheduling and Booking:** It streamlines the appointment booking process, allowing patients to easily view doctor schedules and select convenient slots, thereby reducing waiting times and eliminating double-booking conflicts.
- **Enhanced Payment and Record Keeping:** By integrating secure online payment gateways (Razorpay), the system automates financial transactions and provides clear, auditable records for both patients and administrators.
- **AI-Assisted Preliminary Support:** The integrated AI health assistant offers automated preliminary health information, providing users with immediate access to guidance before formal medical consultations.
- **Authentication and Security:** The system ensures secure access to healthcare services through user registration, email-based OTP verification, and JWT-based authentication, protecting sensitive personal information.
- **Efficiency and Scalability:** By automating routine tasks like doctor onboarding, schedule updates, and appointment confirmation, the system significantly reduces administrative workload and enhances the overall efficiency of clinical staff.
- **Accessibility:** The web-based architecture allows for ubiquitous access, enabling users to manage their healthcare interactions from any device with internet connectivity.

SCOPE OF THE PROPOSED SYSTEM

The current scope of the Smart Healthcare Services Platform focuses on providing a comprehensive digital platform for streamlining hospital administration and patient engagement.

Current System Capabilities

The application encompasses the following modules and functionalities:

1. **User Management and Authentication:** Comprehensive registration and login system with email-based OTP verification and secure JWT-based access control.
2. **Appointment and Schedule Management:** End-to-end management of doctor schedules and patient appointment booking, including status updates and payment integration.
3. **Hospital and Doctor Administration:** Administrative tools for hospital registration, doctor onboarding, and management of specialized profiles.
4. **Interactive AI Assistance:** An AI-powered health assistant capable of processing user queries and providing preliminary health-related information.
5. **Financial Transactions:** Integration of Razorpay for seamless and secure consultation fee payments.
6. **User Engagement:** A robust feedback loop through the review and rating system, allowing patients to share their experience.
7. **Digital Data Management:** Centralized management of hospital, doctor, appointment, and review data using MongoDB.

Potential Applications

The proposed system is designed to be applicable in diverse healthcare settings, including:

- Private clinics and multi-specialty hospitals.
- Small-to-medium healthcare organizations requiring centralized digital management.
- Multi-hospital environments managed by a central administration.

Exclusions (Future Possibilities)

The current version does not include features such as telemedicine/WebRTC, Electronic Health Records (EHR), automated prescription generation, SMS/WhatsApp notifications, AI-based bed resource prediction, or complete server-side role-based access control (RBAC) enforcement. These remain as potential future enhancements to the system.

FACT FINDING TECHNIQUES

1. Interview

Discussions were conducted with domain stakeholders to clarify expectations regarding administrative workflows, appointment management, and the necessary roles within the application. These discussions helped prioritize critical features like OTP verification and payment integration.

2. Observation

Existing clinical workflows were analyzed to understand how appointments are typically managed, enabling the design of a digital dashboard that mimics and improves upon these processes.

3. Questionnaires

Informal feedback sessions were conducted to gauge user expectations regarding the interface, AI assistant utility, and the primary pain points in traditional healthcare booking.

4. Record Searching

Documentation regarding existing hospital management software and industry-standard workflows for scheduling and payment processing was reviewed to ensure the application aligns with best practices.

FEASIBILITY STUDY

1. Technical Feasibility

The project utilizes a modern technology stack that is highly effective for web-based development. The use of FastAPI with Python allows for non-blocking, asynchronous operations which is critical for real-time applications. The backend is powered by Motor (asynchronous MongoDB driver) for efficient database interactions. The React-based frontend ensures a highly responsive user interface, while Docker provides containerized deployment capabilities, ensuring the system runs consistently across various environments.

2. Economic Feasibility

The project prioritizes cost-effective development by leveraging open-source technologies such as Python, FastAPI, React, and MongoDB. The system relies on third-party services like Google Gemini, Razorpay, and SMTP providers, which offer flexible, usage-based scaling. This approach minimizes upfront capital expenditure, making it an economically viable solution for student projects and small organizations.

3. Operational Feasibility

The application is designed to be user-friendly for all stakeholders. Patients benefit from simplified booking and access to information; doctors and hospital administrators gain tools to manage schedules and information efficiently; and super administrators can oversee the platform's overall health. By digitizing manual workflows, the system promotes higher accuracy, better organization, and improved service availability.

4. Schedule Feasibility

The project was developed within a one-month academic timeframe. The development followed a structured progression: requirement analysis and system design, followed by frontend and backend development in parallel, integration of external APIs (Gemini/Razorpay), and concluding with rigorous testing and deployment preparation. This incremental approach ensured all core features were delivered within the project deadline.

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirement:

To support development and smooth operation of the application:

- **Processor:** Multi-core processor (Intel Core i5 or AMD Ryzen 5 recommended, Pentium III minimum).
- **RAM:** 8GB recommended (64MB minimum for basic testing).
- **Storage:** Minimum 10GB free disk space for development environment, database, and Docker containers.
- **Network:** Stable internet connection for API communication and cloud database access.

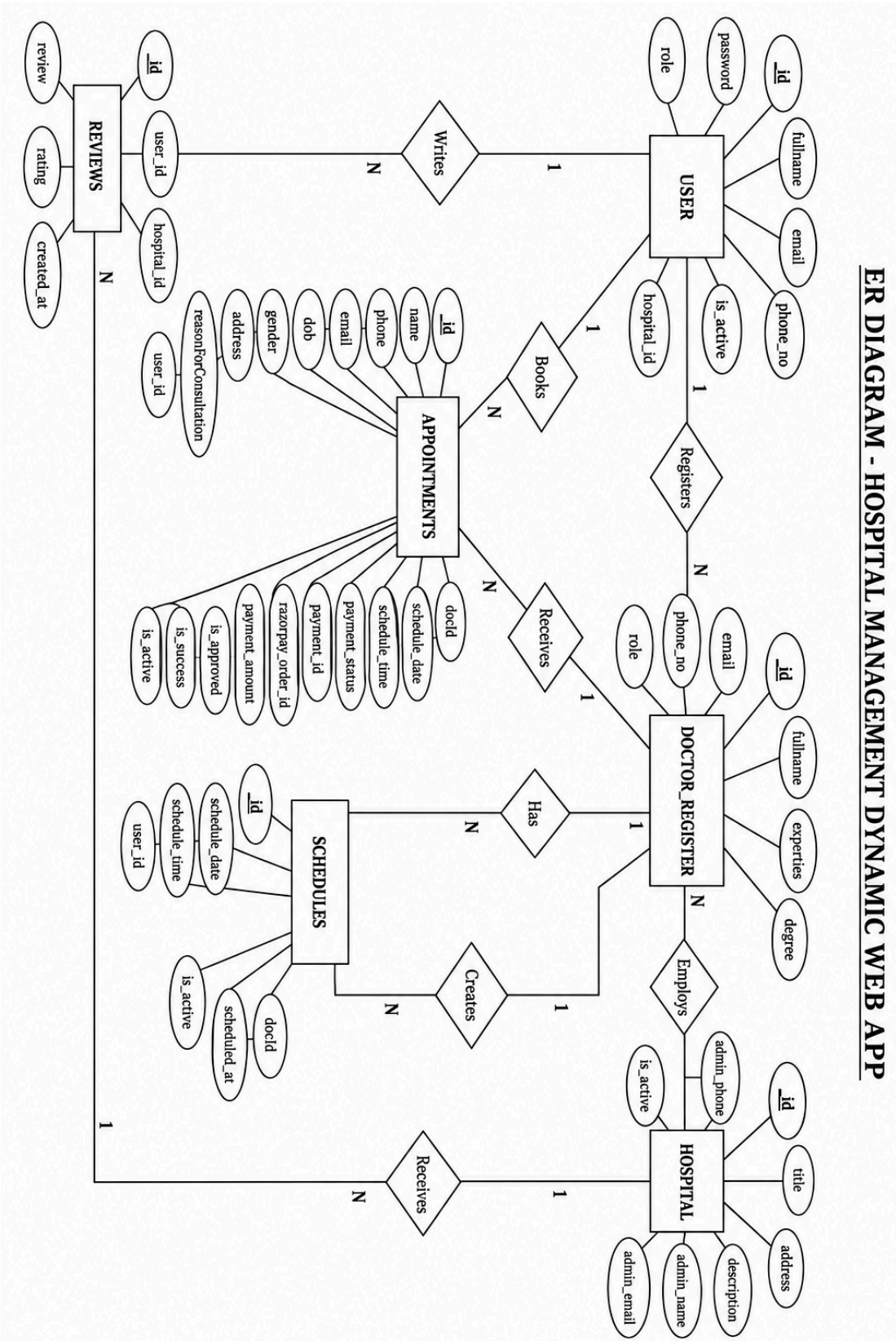
Software Requirement:

The project relies on the following development and runtime environments:

- **Operating System:** Windows 10/11, macOS, or Linux.
- **Backend:** Python 3.12, FastAPI 0.115.12, Pydantic 2.11.4, Motor, PyJWT, Passlib, bcrypt.
- **Frontend:** React 18.3.1, TypeScript, Material UI 7, React Router, Formik, Axios, Node.js/npm.
- **Database:** MongoDB Atlas.
- **Deployment/Infrastructure:** Docker, Docker Compose, Nginx.
- **External Services:** Google Gemini API, Razorpay API, SMTP Email Service.
- **Development Tools:** Visual Studio Code or similar IDE, Modern web browser (Chrome/Firefox/Edge).

ER Diagram

ER DIAGRAM - HOSPITAL MANAGEMENT DYNAMIC WEB APP



11.1 USER COLLECTION

Sr. No.	Column Name	Data Type	Key / Constraint	Description
1	_id	ObjectId	Primary Key	Unique user identifier
2	fullname	String	Required	Full name of the user
3	email	String	Unique	User email address
4	phone_no	String	Required	Contact number
5	password	String	Required	Encrypted password
6	role	String	Required	User role (Patient/Doctor/Admin)

11.2 HOSPITAL COLLECTION

Sr. No.	Column Name	Data Type	Key / Constraint	Description
1	_id	ObjectId	Primary Key	Unique hospital identifier
2	title	String	Required	Name of the hospital
3	address	String	Required	Physical location
4	admin_email	String	Required	Admin contact email

11.3 DOCTOR_REGISTER COLLECTION

Sr. No.	Column Name	Data Type	Key / Constraint	Description
1	_id	ObjectId	Primary Key	Unique doctor identifier
2	fullname	String	Required	Name of the doctor
3	experties	String	Required	Medical specialization
4	hospital_id	ObjectId	Reference	Linked hospital identifier

11.4 SCHEDULES COLLECTION

Sr. No.	Column Name	Data Type	Key / Constraint	Description
1	_id	ObjectId	Primary Key	Unique schedule identifier
2	schedule_date	Date	Required	Date of availability
3	docId	ObjectId	Reference	Linked doctor identifier

11.5 APPOINTMENTS COLLECTION

Sr. No.	Column Name	Data Type	Key / Constraint	Description
1	_id	ObjectId	Primary Key	Unique appointment identifier
2	user_id	ObjectId	Reference	Linked patient identifier
3	docId	ObjectId	Reference	Linked doctor identifier
4	payment_status	String	Required	Status of consultation fee payment
5	razorpay_order_id	String	Reference	Link to Razorpay transaction

11.6 REVIEWS COLLECTION

Sr. No.	Column Name	Data Type	Key / Constraint	Description
1	_id	ObjectId	Primary Key	Unique review identifier
2	user_id	ObjectId	Reference	Linked patient identifier
3	hospital_id	ObjectId	Reference	Linked hospital identifier
4	rating	Integer	Required	Numerical review rating

DATABASE RELATIONSHIPS

- **Hospital to Doctor:** One-to-Many association for administrative oversight.

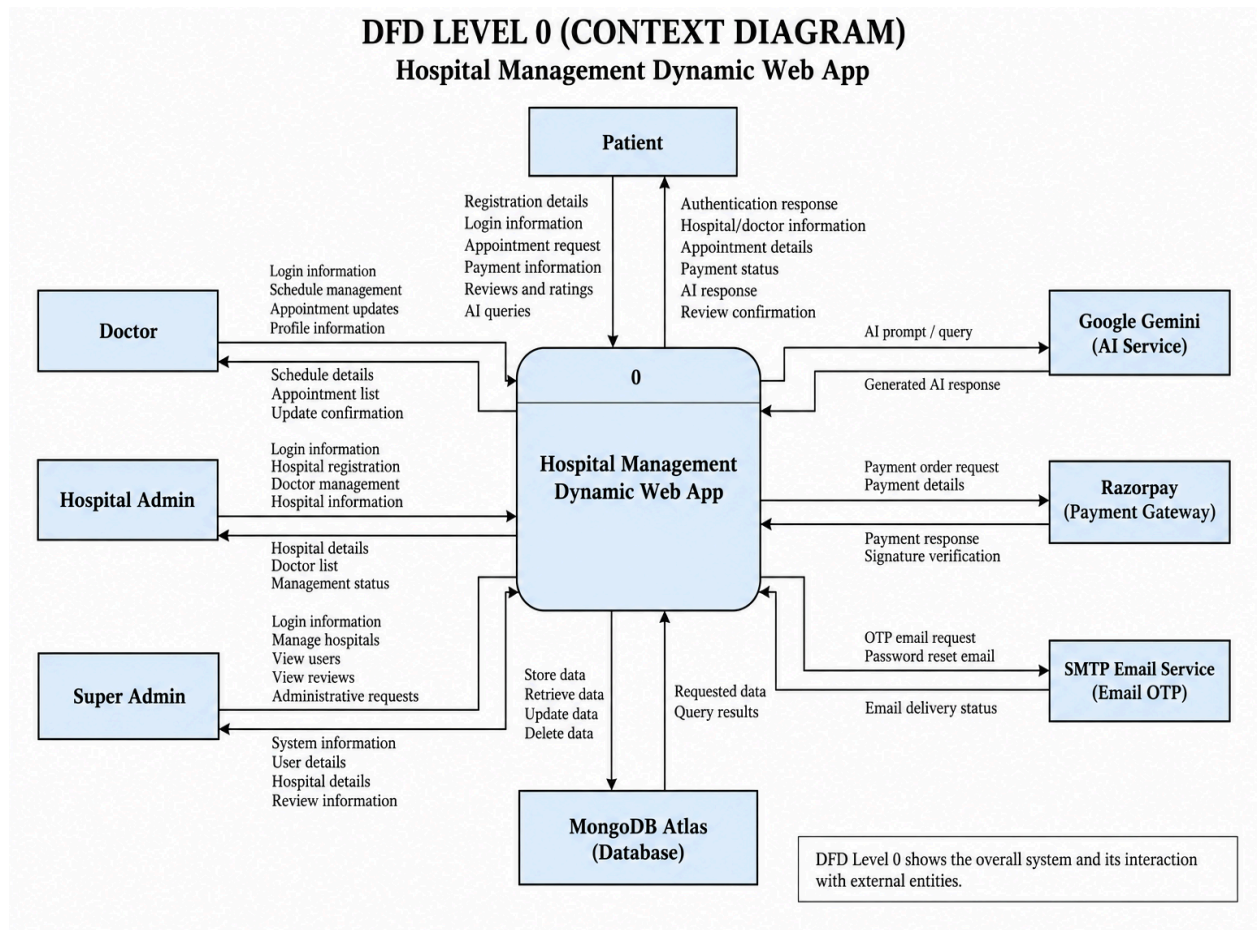
- **Doctor to Schedule/Appointment:** One-to-Many association for management of time and patient visits.
- **User to Appointment/Review:** One-to-Many association for tracking user activity.
- **Payment Information:** Maintained natively within the APPOINTMENT record to ensure transactional integrity.

DFD / UML DIAGRAMS

12.1 DATA FLOW DIAGRAM — LEVEL 0 (CONTEXT DIAGRAM)

The Context Diagram illustrates the interaction between the system and external actors

including Patients, Doctors, Hospital Admins, Super Admins, and external services like Razorpay, Gemini, and SMTP.

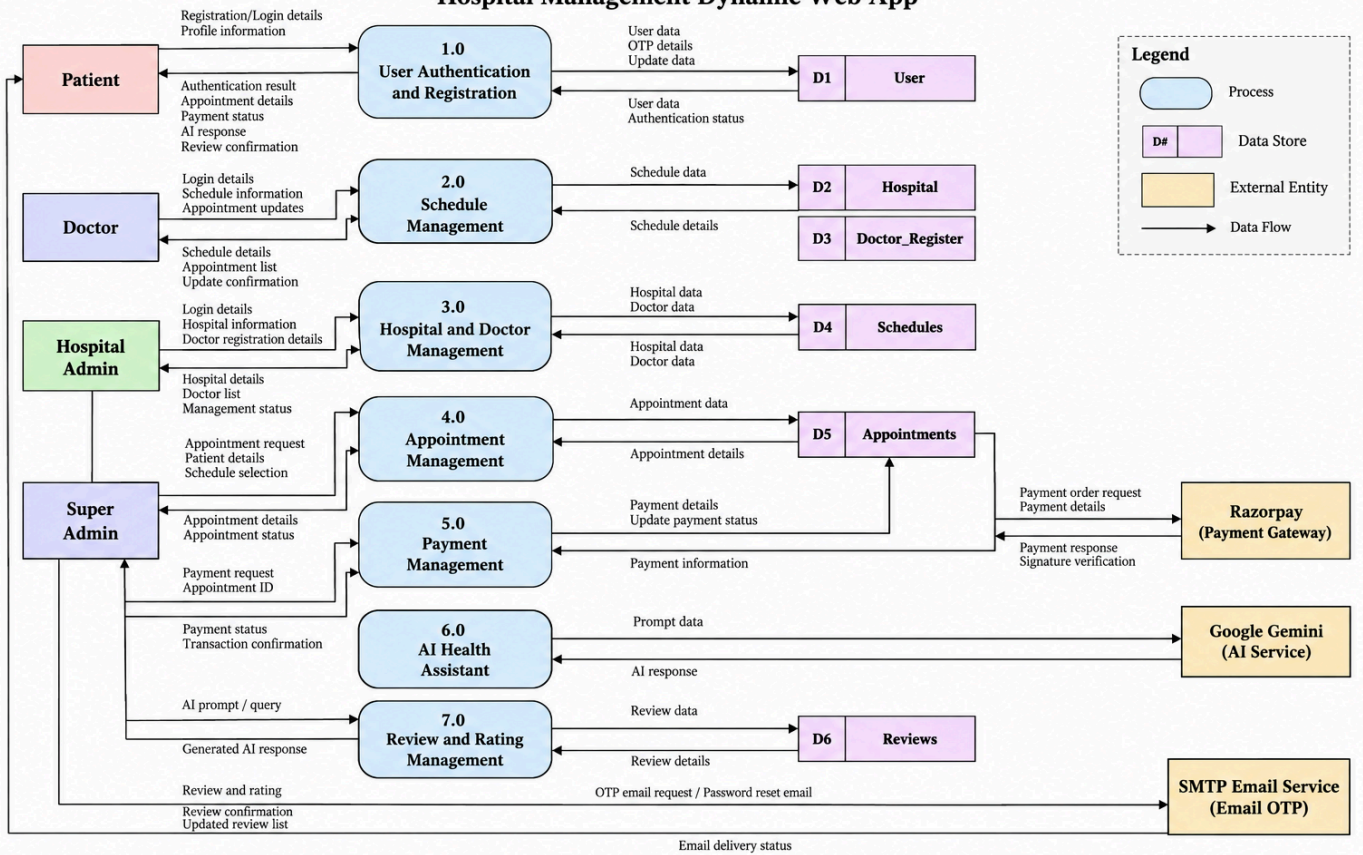


12.2 DFD LEVEL 1

The Level 1 DFD decomposes the system into: (1) User Authentication, (2) Hospital/Doctor Management, (3) Schedule Management, (4) Appointment Management, (5) Payment Management, (6) AI Assistance, and (7) Review Management.

DFD LEVEL 1

Hospital Management Dynamic Web App

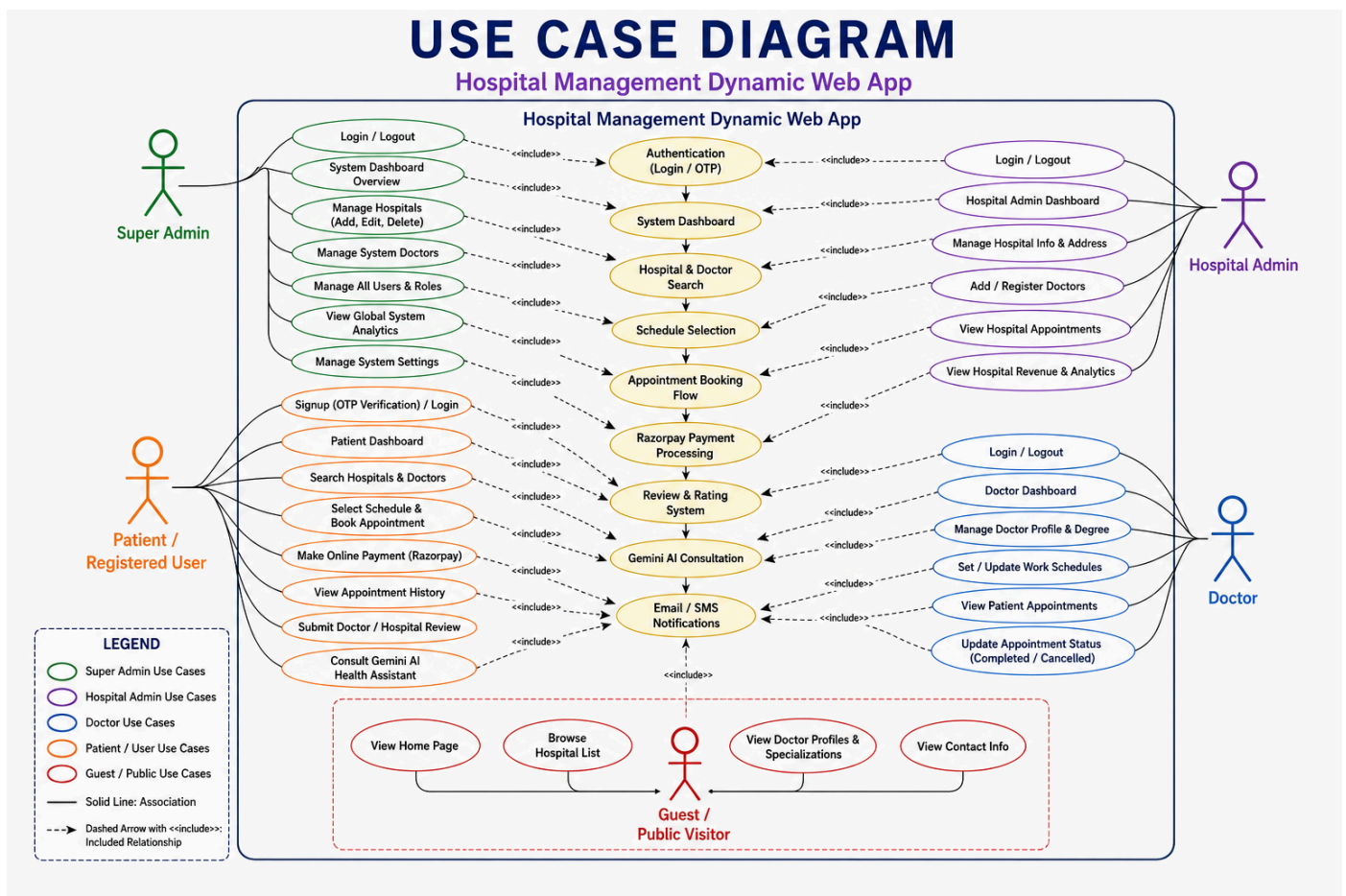


13. UML DIAGRAMS

UML (Unified Modeling Language) diagrams are used to represent the structure, behavior, interactions, and deployment architecture of the Smart Healthcare Services Platform.

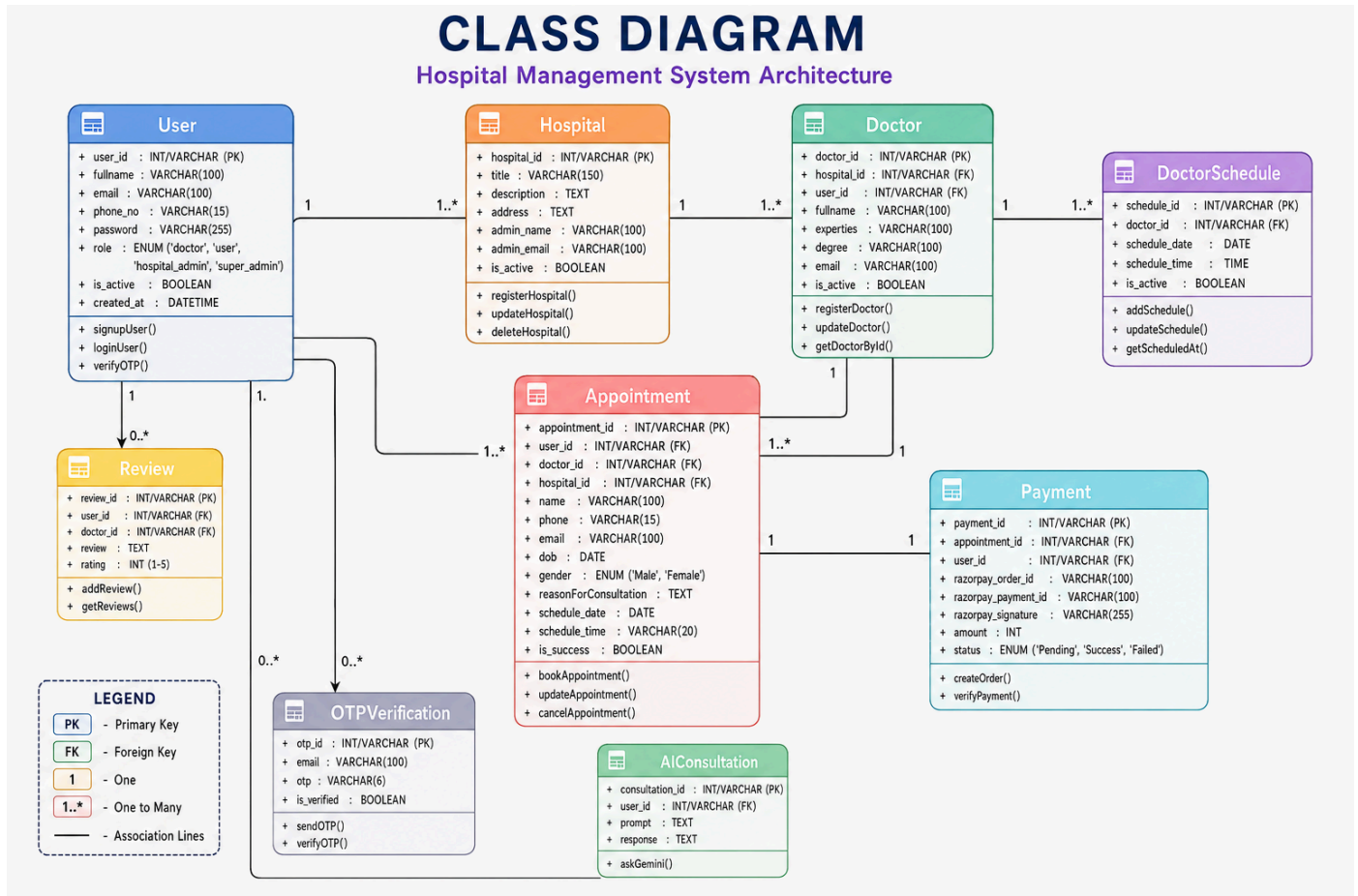
13.1 USE CASE DIAGRAM

The Use Case Diagram represents the functional interactions between the system and its major actors. The primary actors are Patient, Doctor, Hospital Admin, and Super Admin. External services such as Razorpay, Google Gemini, and SMTP Email Service are involved in specific system operations.



13.2 CLASS DIAGRAM

The Class Diagram represents the static structure of the Smart Healthcare Services Platform. It identifies the major classes/entities such as User, Hospital, Doctor, Schedule, Appointment, and Review, along with their attributes and relationships.

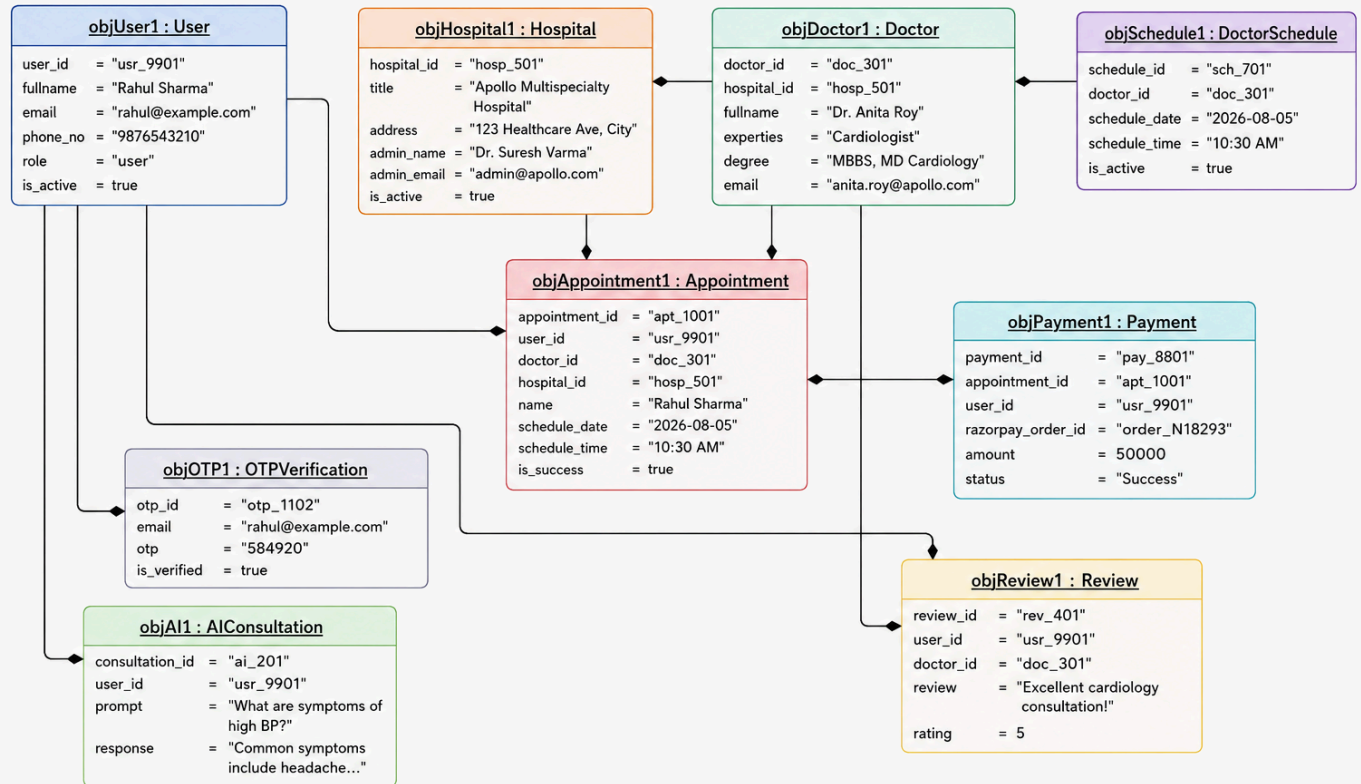


13.3 OBJECT DIAGRAM

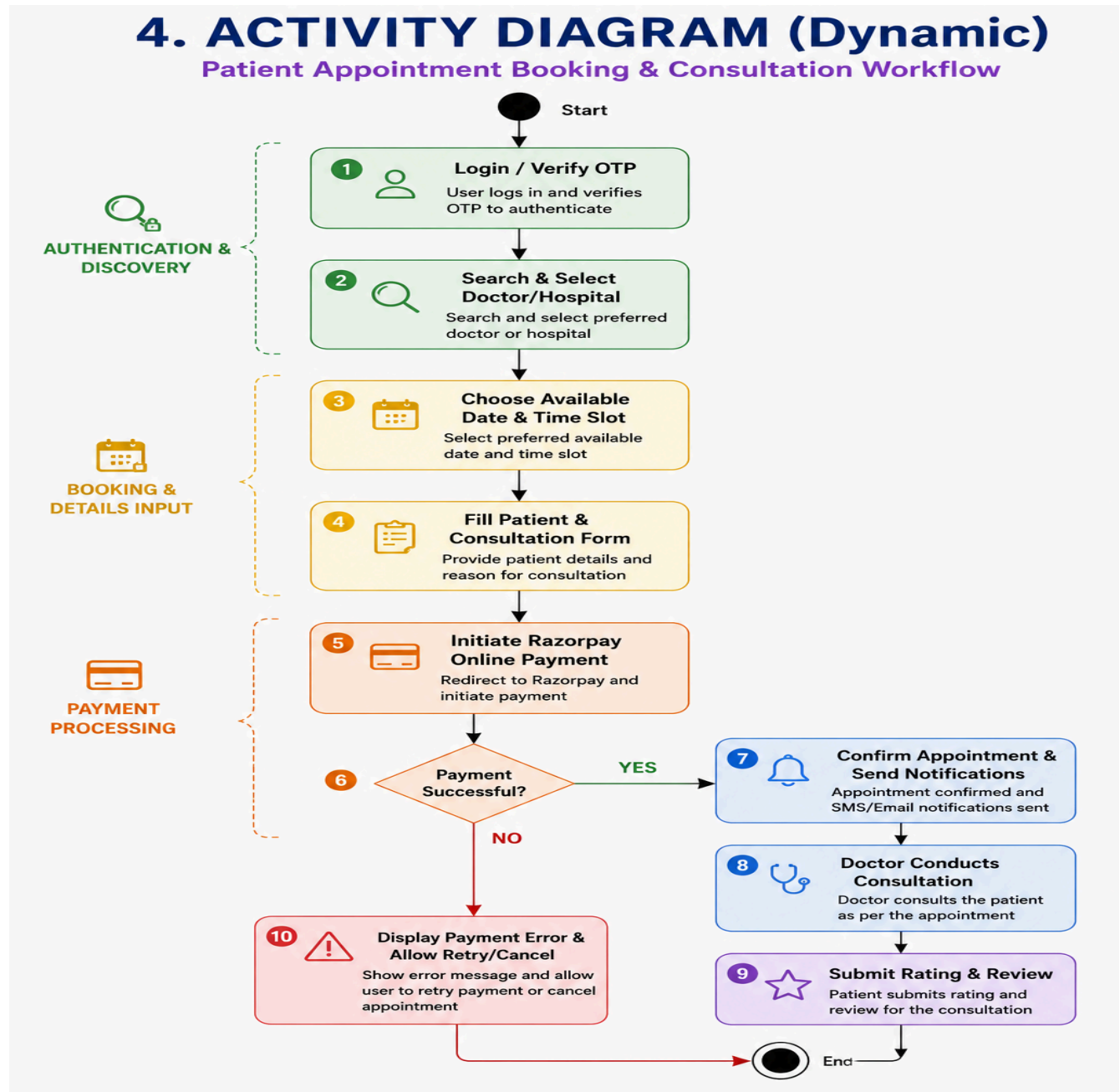
The Object Diagram represents a snapshot of objects and their relationships at a particular point in the system. It illustrates representative instances of User, Hospital, Doctor, Schedule, Appointment, and Review. The objects should use illustrative/example data only and must not contain real patient or user information.

3. OBJECT DIAGRAM (Static)

Runtime Instance Snapshot - Hospital Management System

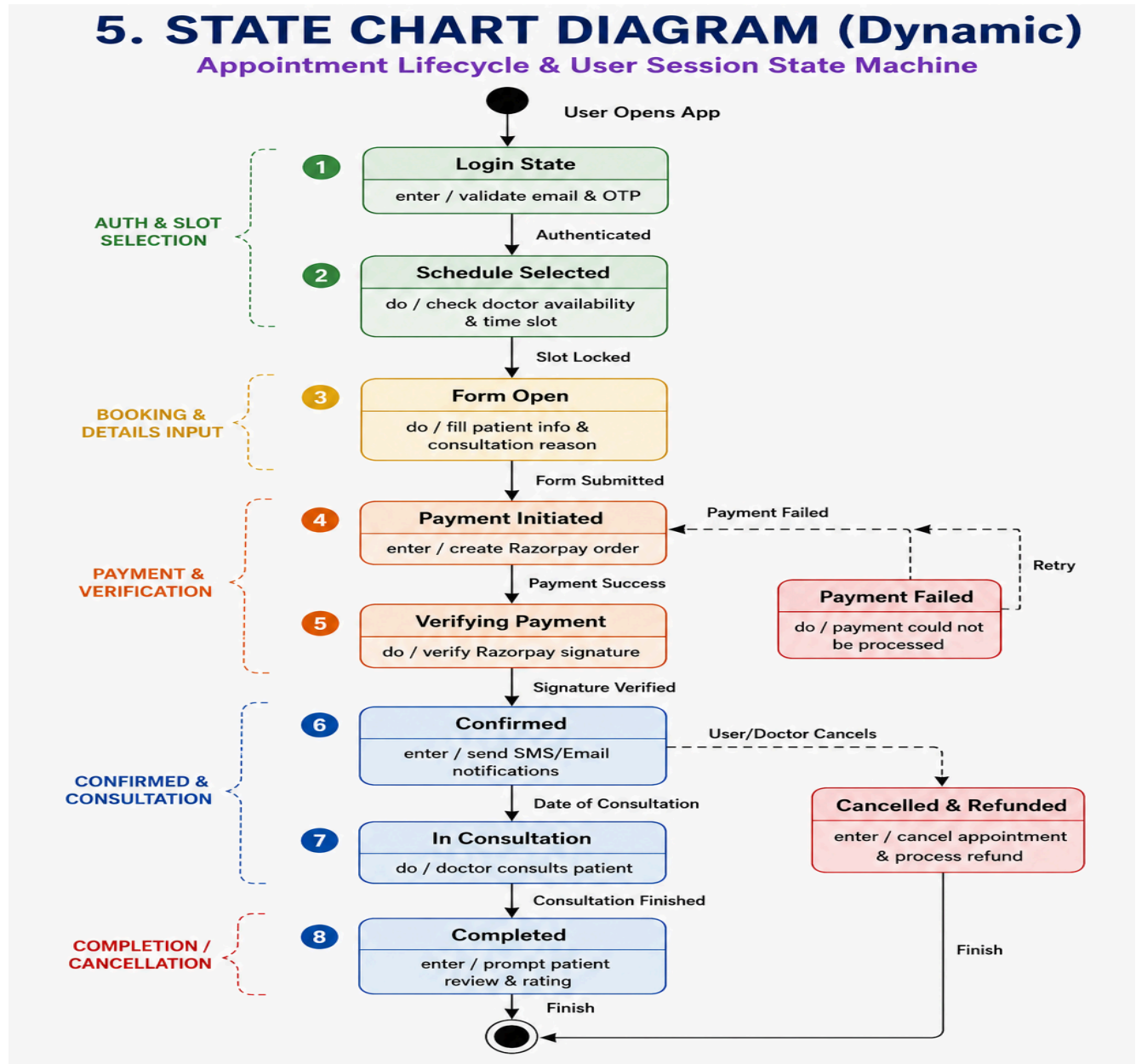


The Activity Diagram represents the workflow of the patient appointment booking and payment process. It illustrates the sequence of activities from authentication and doctor selection through schedule selection, appointment creation, Razorpay payment, payment verification, and appointment processing.



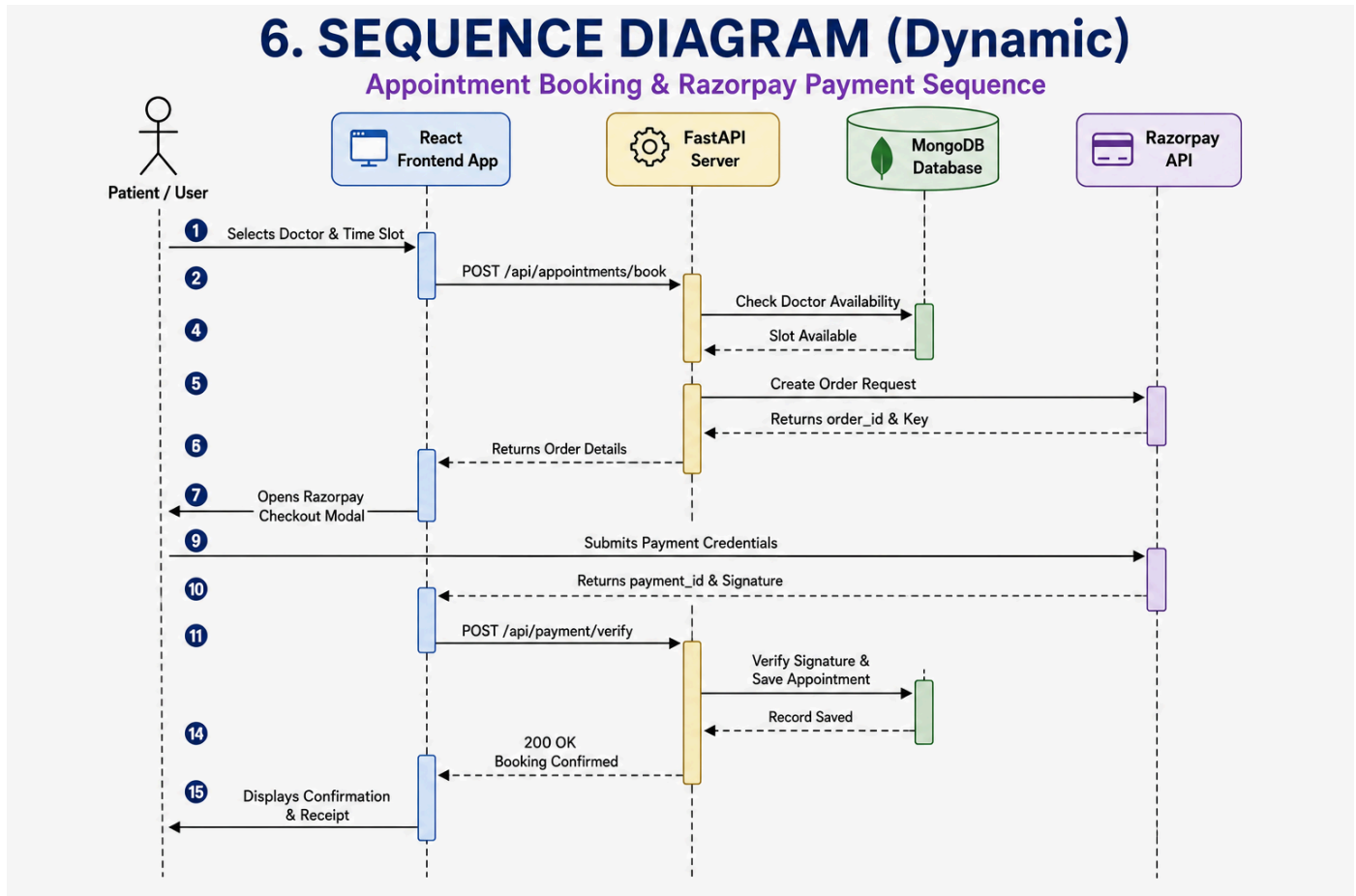
13.5 STATE CHART DIAGRAM

The State Chart Diagram represents the different states of an appointment during its lifecycle, including appointment creation, payment processing, approval, and successful/completed processing.



13.6 SEQUENCE DIAGRAM

The Sequence Diagram represents the interaction between the patient, frontend, backend, database, and payment gateway in chronological order. The diagram illustrates the appointment booking and online payment workflow of the Smart Healthcare Services Platform.

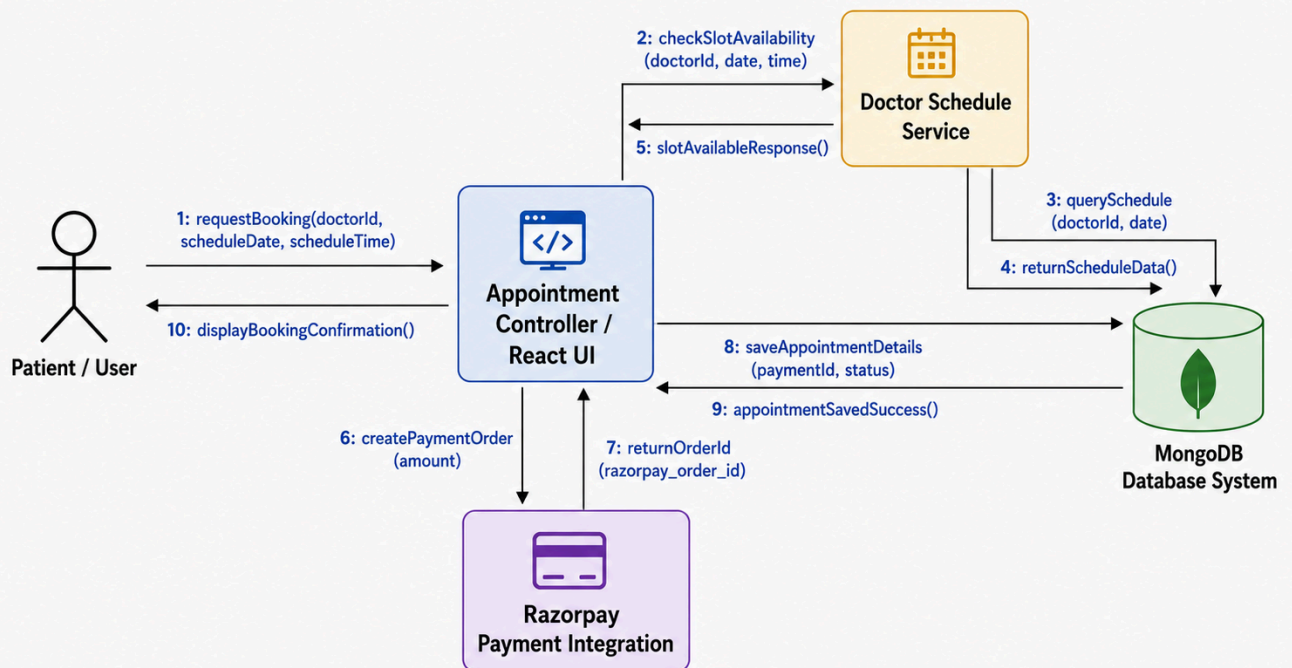


13.7 COLLABORATION / COMMUNICATION DIAGRAM

The Collaboration or Communication Diagram represents how different objects or components communicate with one another to complete a system operation. For this project, the Patient Appointment and Payment workflow should be represented using the Patient, Frontend, FastAPI Backend, MongoDB, and Razorpay.

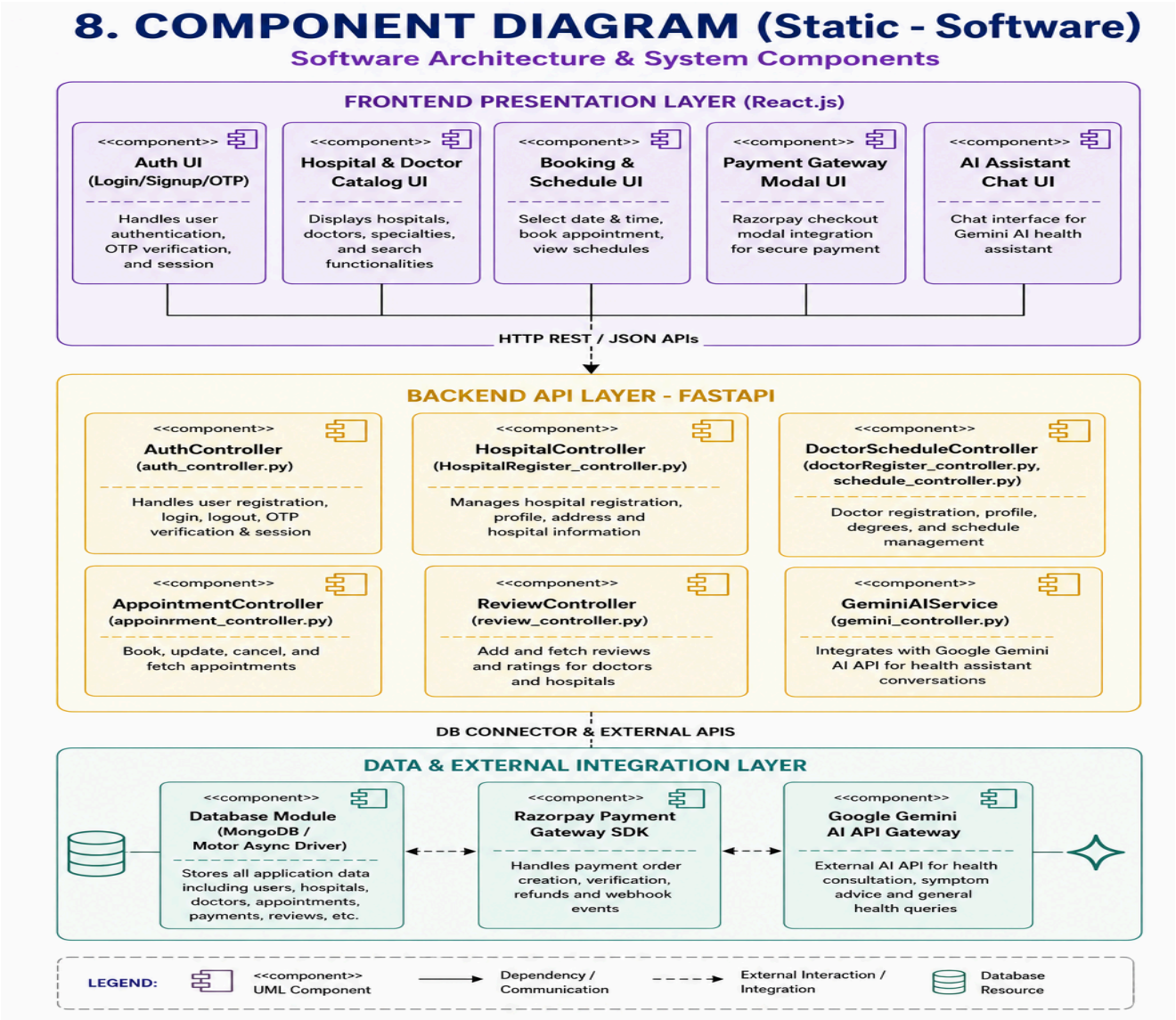
7. COLLABORATION DIAGRAM (COMMUNICATION DIAGRAM) (Dynamic)

Object Communication & Message Exchange Network



13.8 COMPONENT DIAGRAM

The Component Diagram represents the high-level software architecture of the Smart Healthcare Services Platform. Major components include the React frontend, Axios API client, FastAPI backend, routes, controllers, services, authentication middleware, MongoDB Atlas, Razorpay, Google Gemini, and SMTP Email Service.

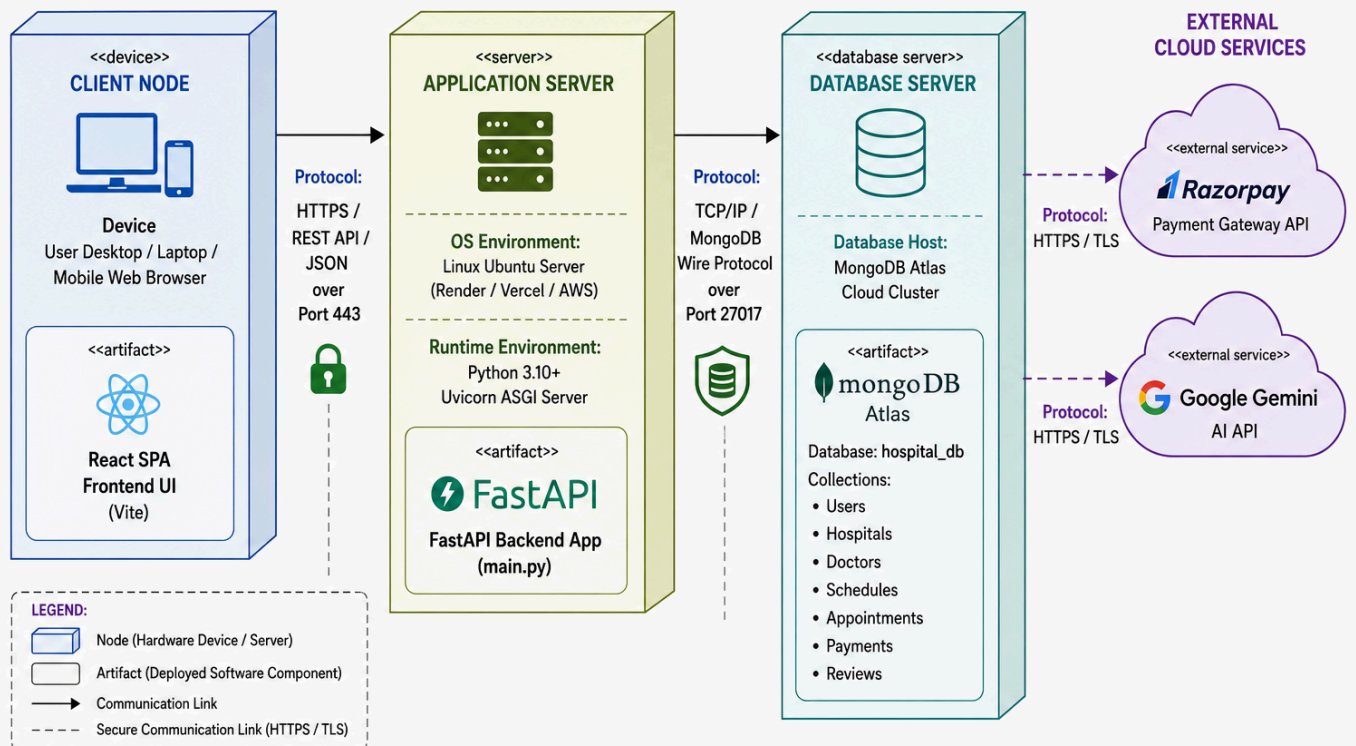


13.9 DEPLOYMENT DIAGRAM

The Deployment Diagram represents the physical and logical deployment architecture of the application. The system uses Docker-based deployment consisting of the client browser, frontend/Nginx container, backend/FastAPI container, and MongoDB Atlas, with external communication to Razorpay, Google Gemini, and SMTP Email Service.

9. DEPLOYMENT DIAGRAM (Static - Hardware)

Hardware Infrastructure & Deployment Nodes



INPUT OUTPUT SCREENS

This section catalogs the primary user interfaces enabling interaction with the Smart Healthcare Services Platform.

Screen Name	Purpose	User	Important Functionality
Home Page	Initial entry point	All Users	Navigation, branding
Registration	Create account	Patient	OTP verification
Login	Access dashboard	All Users	JWT validation
Doctor List	View available doctors	Patient	Searching/Filtering
Booking Form	Schedule appointment	Patient	Razorpay integration
Appointment Status	Track bookings	Patient	Viewing payment status
Admin Dashboard	Management overview	Admin	Hospital/Doctor ops
AI Chat	Preliminary info	Patient	Gemini integration

RECOMMENDED FINAL SCREENSHOT SET:

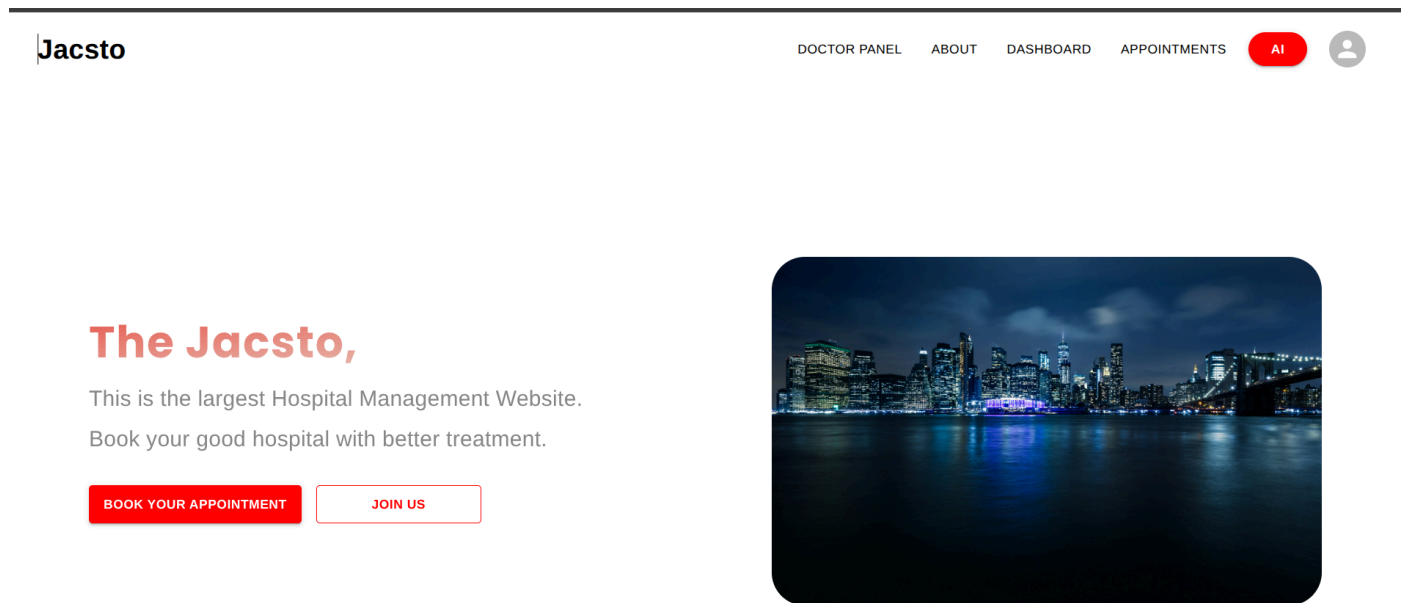
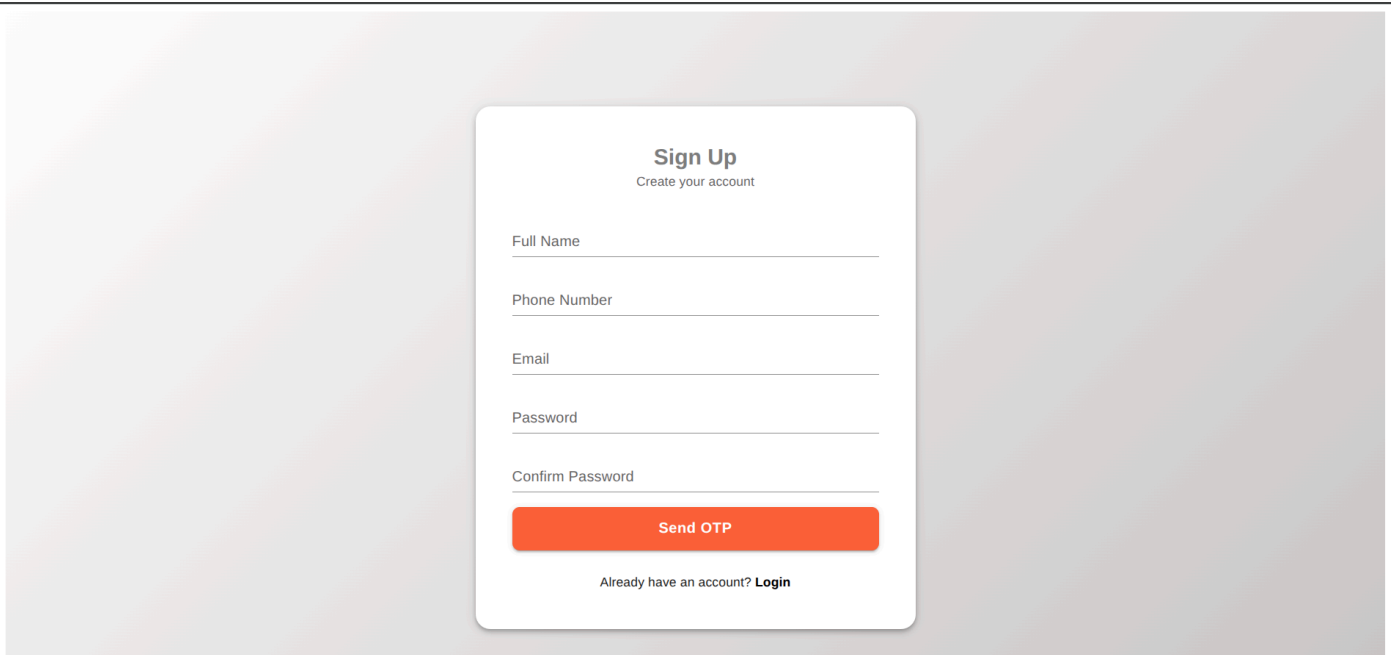


Figure 14.1: Home Page



The registration form is centered on a light gray background. It features a white card with rounded corners. At the top, the title "Sign Up" is in bold, followed by the subtitle "Create your account". Below this are five input fields: "Full Name", "Phone Number", "Email", "Password", and "Confirm Password". Each field has a thin gray border and a small gray line at the bottom. A prominent orange button with the text "Send OTP" is positioned below the input fields. At the bottom of the card, there is a link that says "Already have an account? Login".

Sign Up
Create your account

Full Name

Phone Number

Email

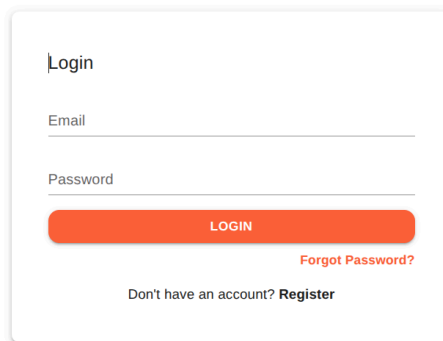
Password

Confirm Password

Send OTP

Already have an account? [Login](#)

Figure 14.2: User Registration



The login form is centered on a light gray background. It features a white card with rounded corners. At the top, the title "Login" is in bold. Below this are two input fields: "Email" and "Password". Each field has a thin gray border and a small gray line at the bottom. A prominent orange button with the text "LOGIN" is positioned below the input fields. Below the button, there is a link that says "Forgot Password?". At the bottom of the card, there is a link that says "Don't have an account? Register".

Login

Email

Password

LOGIN

[Forgot Password?](#)

Don't have an account? [Register](#)

Figure 14.3: Login Page

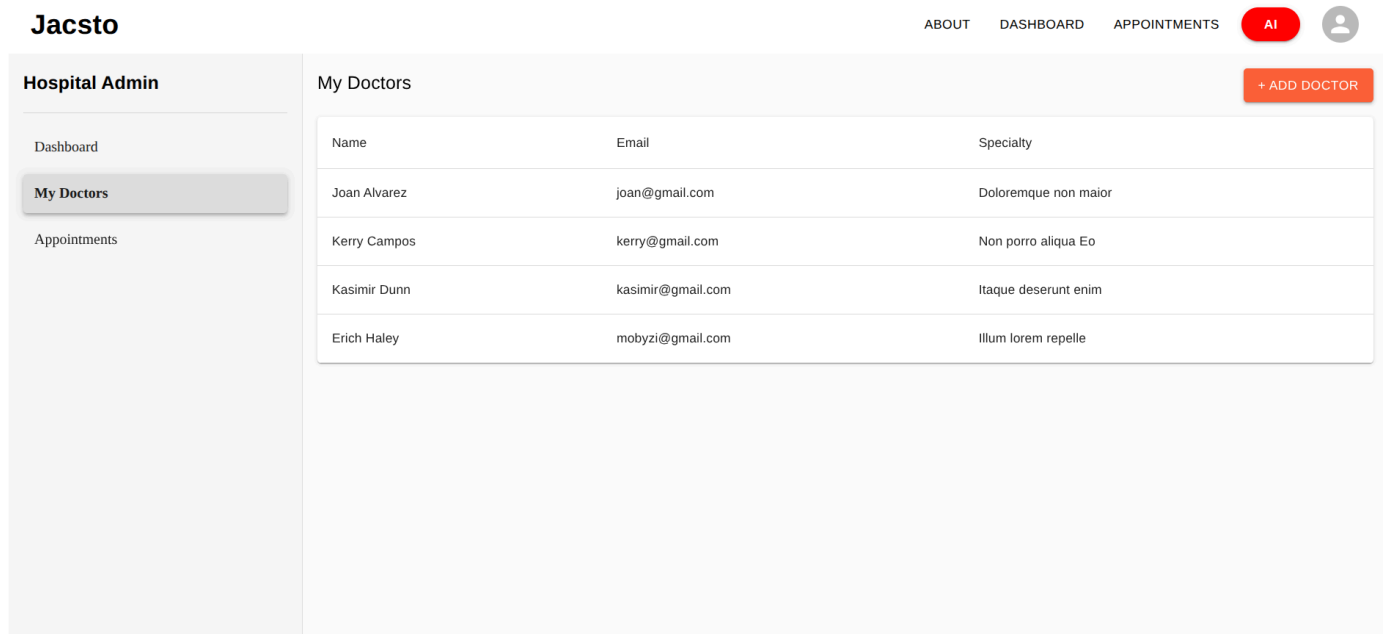


Figure 14.4: Doctor List

Jacsto

ABOUT DASHBOARD APPOINTMENTS AI

Confirm Appointment

Date
2026-12-23

Time
05:45 PM

Name

Phone

Email

DOB
mm/dd/yyyy

Gender

Address

Reason for Consultation

Fee: ₹499 (Pay after doctor approval)

Figure 14.5: Appointment Form

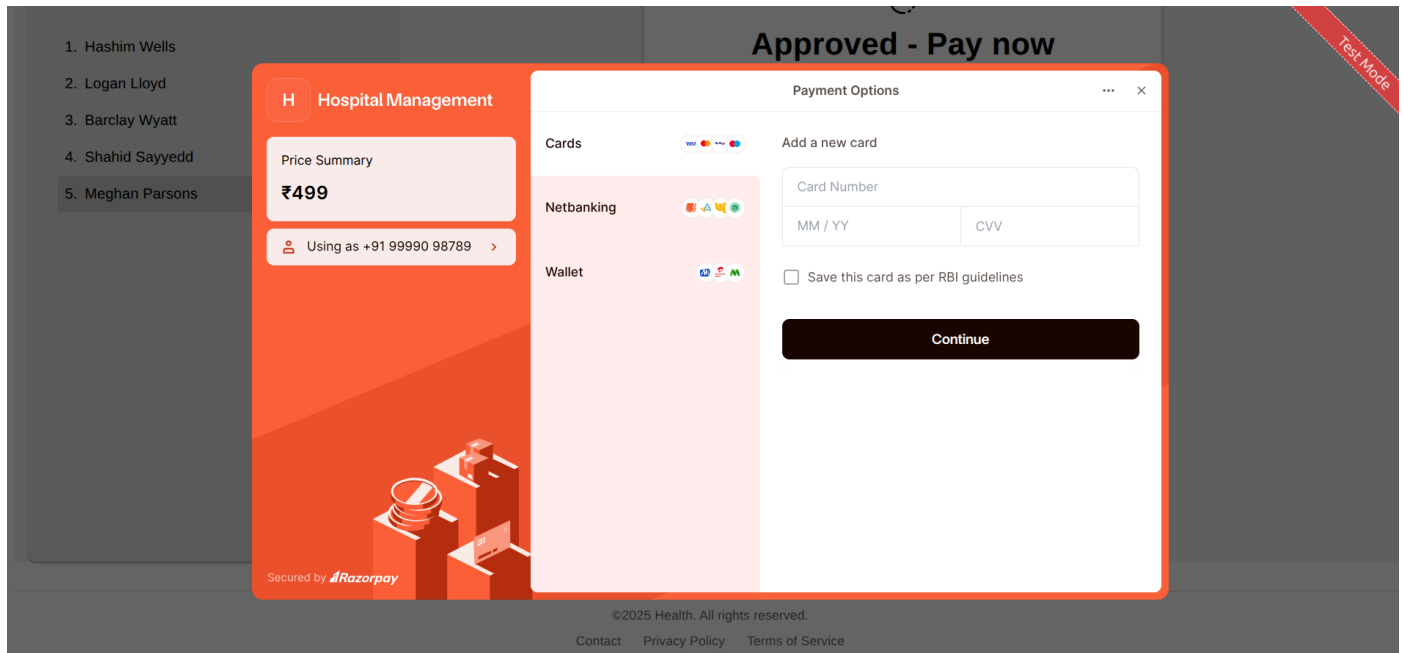


Figure 14.6: Razorpay Payment

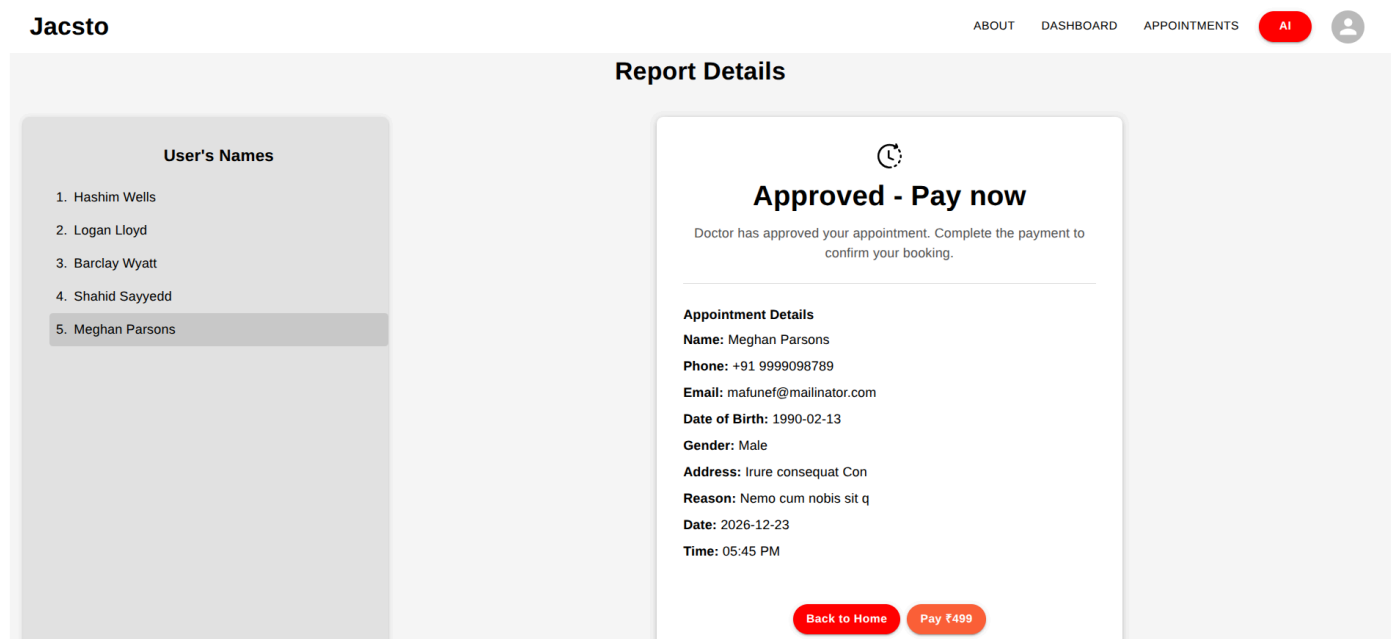


Figure 14.7: Patient Dashboard

Jacsto

DOCTOR PANELABOUTDASHBOARDAPPOINTMENTS

AI

Doctor Panel

Dashboard

Appointments

My Schedule

My Appointments

Patient	Email	Date	Time	Status	Action
Shahid Sayyed	shahidsayyed02w12@gmail.com	2026-12-23	05:45 PM	Booked	
Meghan Parsons	mafunef@mailinator.com	2026-12-23	05:45 PM	Approved	COMPLETE

Figure 14.8: Doctor Dashboard

Jacsto

DOCTOR PANELABOUTDASHBOARDAPPOINTMENTS

AI

Doctor Panel

Dashboard

Appointments

My Schedule

My Schedule

ADD SCHEDULE

Schedule Date	Schedule Time	Actions
2026-12-23	05:45 PM	

Figure 14.9: Schedule Management

How can I assist you? Erich Haley

Type your message...



Figure 14.10: AI Health Assistant

LIMITATIONS

Although the Smart Healthcare Services Platform fulfills its primary objectives of digital hospital administration and appointment management, several technical and functional limitations have been identified during the development and testing phases. These areas are recognized as opportunities for future expansion and refinement.

8. **Role-Based Authorization:** While the frontend implements route guarding for different user roles, comprehensive server-side role-based access control (RBAC) at the backend controller level is currently incomplete.
9. **Concurrency Management:** Real-time appointment slot locking is not implemented. Consequently, concurrent booking attempts may occasionally result in appointment conflicts.
10. **Backend API Coverage:** Although doctor deletion functionality exists within the frontend workflows, the corresponding backend endpoint remains unimplemented.
11. **Appointment Workflow:** The current implementation supports appointment creation and payment, but a comprehensive automated rejection or cancellation workflow is pending future development.
12. **AI Integration:** The integrated AI health assistant provides preliminary information, but conversation history is not currently persisted, and safety filtering is basic.
13. **Administrative Reporting:** Super Administrator reporting capabilities are currently in a basic state.
14. **Security and Configuration:** The application is an academic project and requires further hardening, including more granular CORS configurations, implementation of rate limiting, and improved secret management practices.
15. **Deployment and Testing:** The project utilizes Docker for containerization but currently lacks a CI/CD pipeline and an automated test suite.
16. **Infrastructure:** File upload handling is functional but requires more robust validation for file types and sizes.

CONCLUSION

The Smart Healthcare Services Platform project successfully demonstrates a functional, multi-tenant web-based solution for hospital administration. By utilizing a modern technology stack—comprising React for the frontend, FastAPI for the backend, and MongoDB for data storage—the application effectively streamlines key healthcare workflows.

The developed platform enables patients to securely register, browse available doctors, view schedules, book appointments, and complete payments via Razorpay. Simultaneously, it provides hospital administrators and doctors with essential tools for onboarding, schedule management, and appointment processing. The integration of an AI-based health assistant, powered by Google Gemini, offers users accessible preliminary health information, enhancing the overall utility of the platform.

While the system is functional and meets the requirements for a semester-level project, it is acknowledged as a foundational implementation rather than a complete enterprise hospital information system. The current limitations, such as the need for comprehensive server-side RBAC, automated testing, and refined concurrency handling, highlight specific areas for ongoing development. Future iterations could integrate telemedicine capabilities, electronic health records (EHR), SMS notification systems, and enhanced AI-driven resource predictions to further scale the application's functionality and reliability. Ultimately, this project establishes a scalable digital framework that reduces manual administrative overhead and serves as a viable model for digital healthcare management.

BIBLIOGRAPHY

17. **React Documentation** (v18.3.1) - <https://react.dev/>
18. **TypeScript Documentation** - <https://www.typescriptlang.org/docs/>
19. **Material UI (MUI) Documentation** - <https://mui.com/>
20. **FastAPI Documentation** (v0.115.12) - <https://fastapi.tiangolo.com/>
21. **Python Documentation** (v3.12) - <https://docs.python.org/3/>
22. **MongoDB Atlas Documentation** - <https://www.mongodb.com/docs/atlas/>
23. **Motor (Asynchronous MongoDB Driver) Documentation** - <https://motor.readthedocs.io/>
24. **Docker Documentation** - <https://docs.docker.com/>

- 25. **Nginx Documentation** - <https://nginx.org/en/docs/>
- 26. **Razorpay API Documentation** - <https://razorpay.com/docs/api/>
- 27. **Google Gemini API Documentation** - <https://ai.google.dev/docs>
- 28. **Pydantic Documentation** (v2.11.4) - <https://docs.pydantic.dev/>
- 29. **Axios Documentation** - <https://axios-http.com/docs/intro>
 - **React Router Documentation** - <https://reactrouter.com/en/main>